



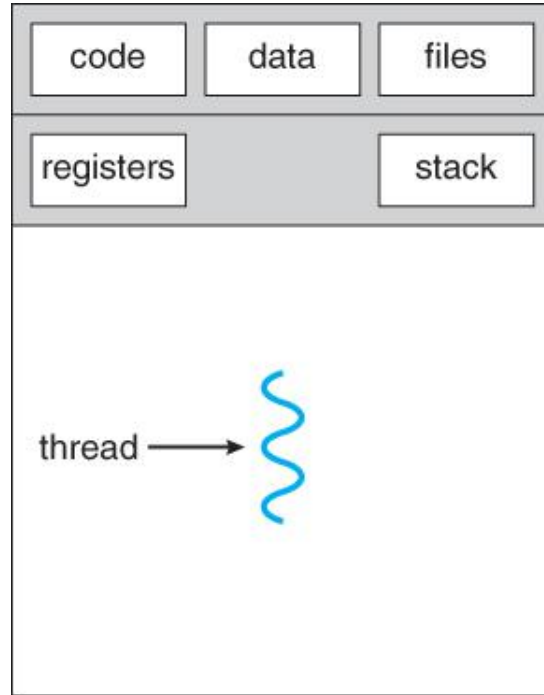
Operating System: Thread

Dr. Jit Mukherjee

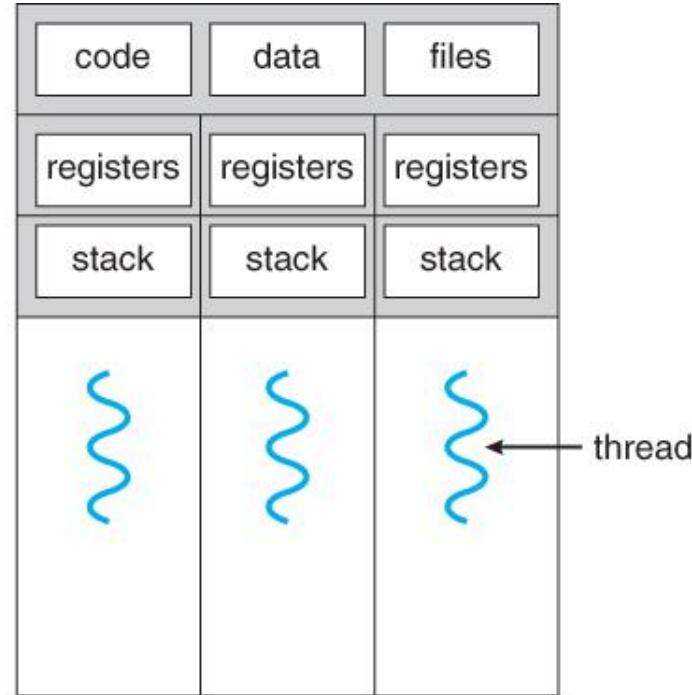


Thread

- A thread is a lightweight process.
- To achieve parallelism by dividing a process into multiple threads.
- A **thread** consists of a program counter, a stack, and a set of registers, (and a thread ID.)
- Multi-threaded applications have multiple threads within a single process, each having their own program counter, stack and set of registers, but sharing common code, data, and certain structures such as open files.



single-threaded process



multithreaded process

Thread: Motivation

- Context Switching leads to an overhead cost sharing the cache between multiple tasks, running the task scheduler etc.
 - Context switching between two threads of the same process is faster than between two different processes as threads have the same virtual memory maps.
- Threads are very useful in modern programming whenever a process has multiple tasks to perform independently of the others.
- This is particularly true when one of the tasks may block, and it is desired to allow the other tasks to proceed without blocking.
- For example in a word processor,
 - a background thread may check spelling and grammar
 - a foreground thread processes user input (keystrokes)
 - a third thread loads images from the hard drive
 - And a fourth does periodic automatic backups of the file being edited.
- Multithreading: a single program made up of a number of different concurrent activities
- A thread shares: code section, data section and OS resources (open files, signals)
- Collectively called a task.
- Heavyweight process is a task with one thread.
- Thread context switch still requires
- set switch, but no memory management related work!!

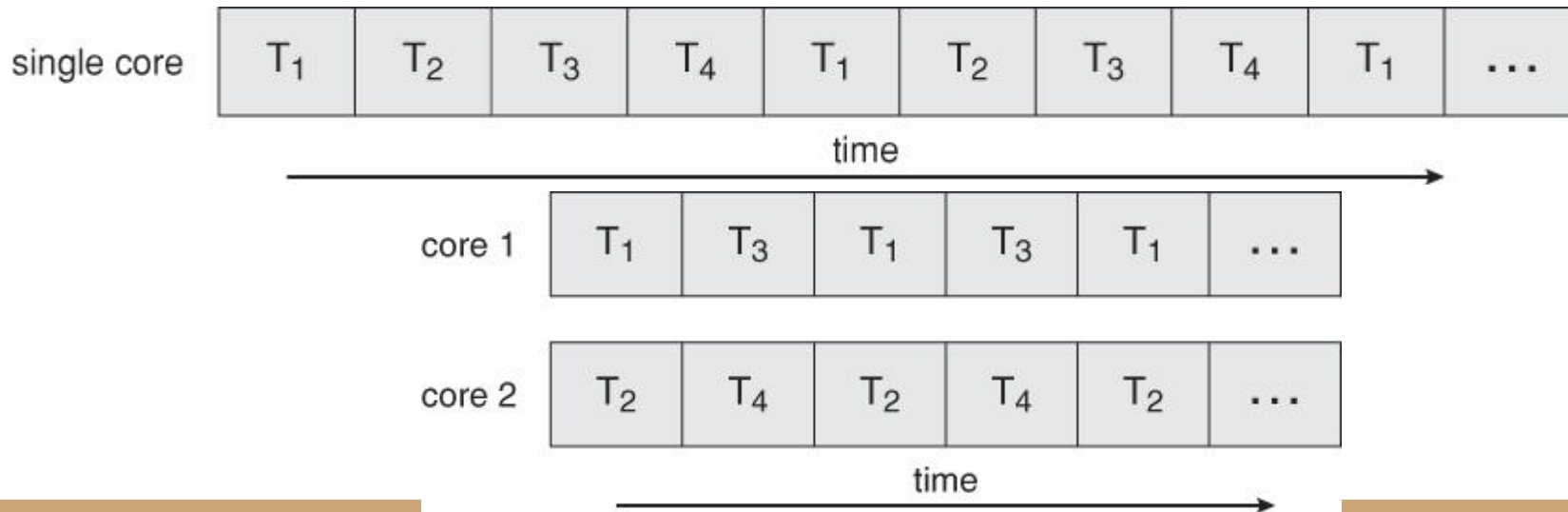
Thread: Benefits

There are four major categories of benefits to multi-threading:

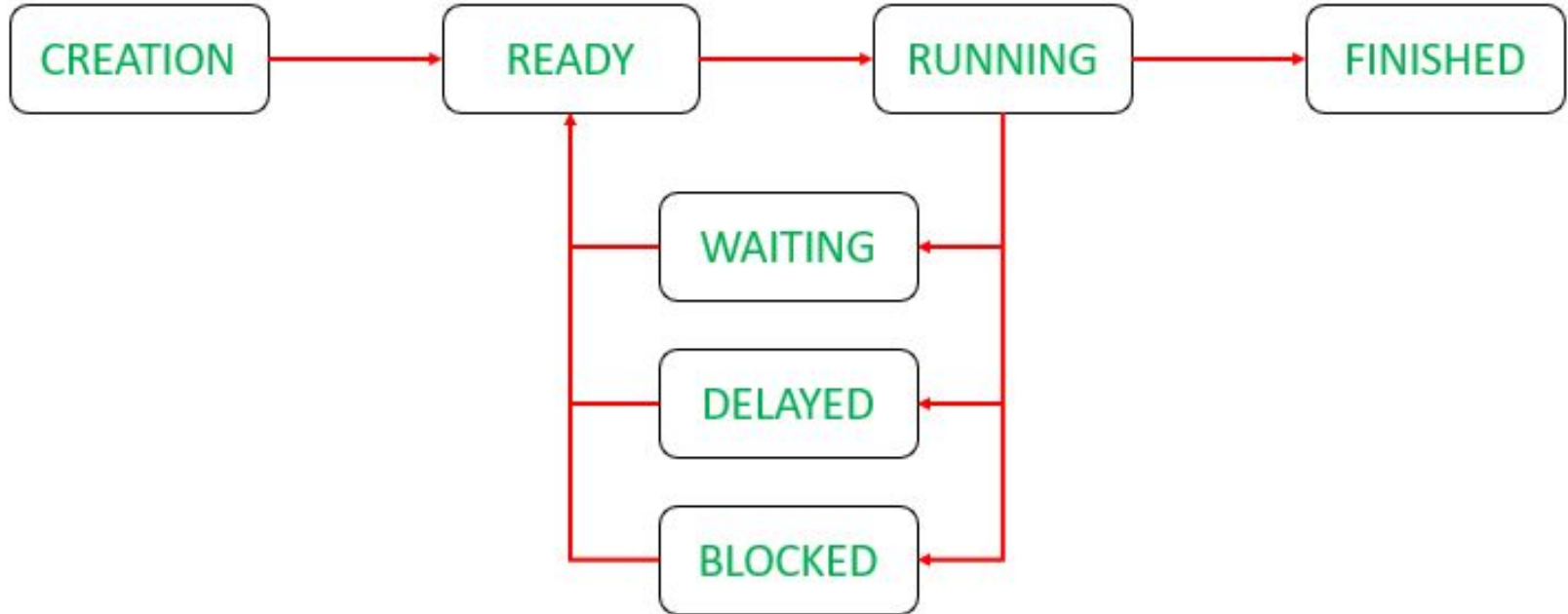
- Responsiveness - One thread may provide rapid response while other threads are blocked or slowed down doing intensive calculations.
- Resource sharing - By default threads share common code, data, and other resources, which allows multiple tasks to be performed simultaneously in a single address space.
- Economy - Creating and managing threads (and context switches between them) is much faster than performing the same tasks for processes.
- Scalability, i.e. Utilization of multiprocessor architectures - A single threaded process can only run on one CPU, no matter how many may be available, whereas the execution of a multi-threaded application may be split amongst available processors. (Note that single threaded processes can still benefit from multi-processor architectures when there are multiple processes contending for the CPU, i.e. when the load average is above some certain threshold.)

Multicore Programming

- A recent trend in computer architecture is to produce chips with multiple *cores*, or CPUs on a single chip.
- A multi-threaded application running on a traditional single-core chip would have to interleave the threads. On a multi-core chip, however, the threads could be spread across the available cores, allowing true parallel processing.
- For operating systems, multi-core chips require new scheduling algorithms to make better use of the multiple cores available.
- As multi-threading becomes more pervasive and more important (thousands instead of tens of threads), CPUs have been developed to support more simultaneous threads per core in hardware.
- Data parallelism and Task parallelism



Life Cycle of Thread



Life Cycle of Thread

- When an application is to be processed, then it creates a thread. It is then allocated the required resources(such as a network) and it comes in the **READY** queue.
- When the thread scheduler (like a process scheduler) assign the thread with processor, it comes in **RUNNING** queue.
- When the process needs some other event to be triggered, which is outside its control (like another process to be completed), it transitions from **RUNNING** to **WAITING** queue.
- When the application has the capability to delay the processing of the thread, it when needed can delay the thread and put it to sleep for a specific amount of time. The thread then transitions from **RUNNING** to **DELAYED** queue.
- When thread generates an I/O request and cannot move further till it's done, it transitions from **RUNNING** to **BLOCKED** queue.
- After the process is completed, the thread transitions from **RUNNING** to **FINISHED**.

Thread Control Block

- **Thread ID:** It is a unique identifier assigned by the Operating System to the thread when it is being created.
- **Thread states:** These are the states of the thread which changes as the thread progresses through the system
- **CPU information:** It includes everything that the OS needs to know about, such as how far the thread has progressed and what data is being used.
- **Thread Priority:** It indicates the weight (or priority) of the thread over other threads which helps the thread scheduler to choose next from the READY queue.
- A **pointer** which points to the process which triggered the creation of this thread.
- A **pointer** which points to the thread(s) created by this thread.

Thread ID
Thread state
CPU information : Program counter Register contents
Thread priority
Pointer to process that created this thread
Pointer(s) to other thread(s) that were created by this thread

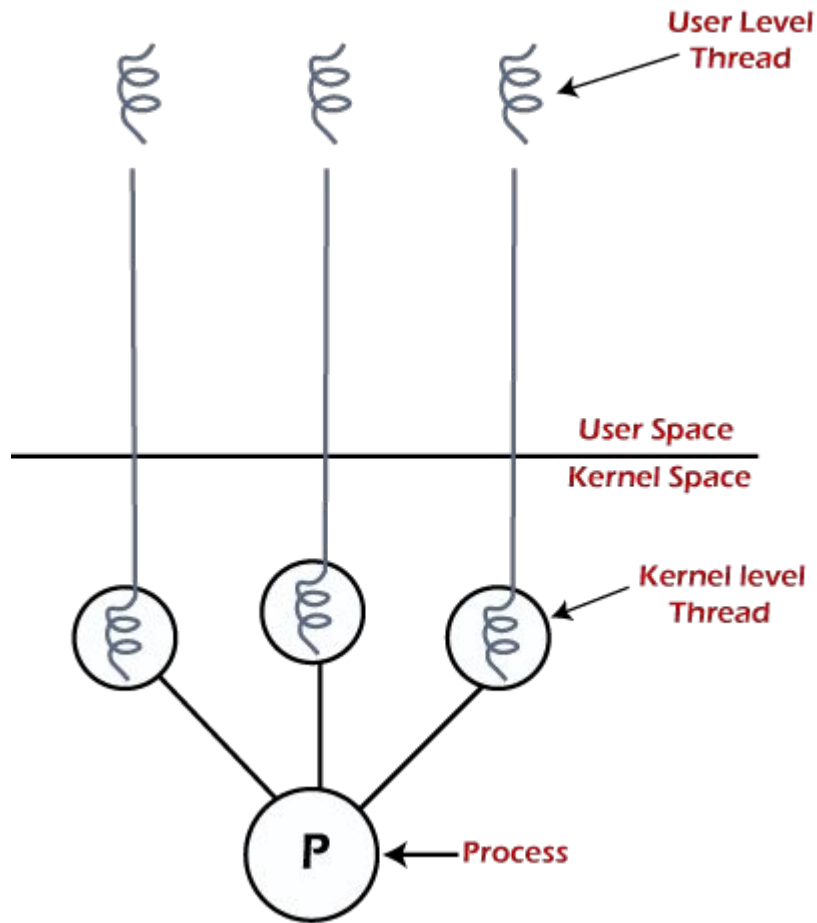
Thread Control Block

- `void * stack` - This field is a pointer to the stack of the thread.
- `u32_t executedTime` - This field can be used for keeping profiling information.
- `u32_t sched_field` - for use by the scheduler object. The kernel never accessed this field. Typically this field will be used by the scheduler object for constructing datastructure of tcb's. For example if the scheduler object stores the ready threads in a list, this field would be used as next pointer.
- `u32_t magic_key` - This field is used for debugging.
- `struct reg_context thread_context` - This structure stores the context of a thread. The structure `reg_context` is architecture specific. This field is accessed by the kernel thread library only. (The scheduler object should not mess with it).
- `struct thread_info thread_params` - This field holds the initial thread parameters, like the start function, stack size, deadline etc. This information is required for resetting threads.
- `unsigned int status` - This field holds the status information of the current thread. It can be one of `THREAD_ON_CPU`, `THREAD_READY`

```
struct tcb {  
    u32_t status;  
    struct reg_context thread_context;  
    void *stack;  
    struct thread_info thread_params;  
    u32_t executedTime;  
    struct tcb *recoveryTask;  
    u32_t sched_field;  
    u32_t magic_key;  
};
```

Types of Threads

- Component of Thread
 - Program counter
 - Register set
 - Stack space
- Kernel-supported threads - User managed threads.
- User-level threads - Operating System managed threads acting on kernel, an operating system core.
- Hybrid approach implements both user-level and kernel-supported threads (Solaris 2).



User Level Thread

- The thread management kernel is not aware of the existence of threads.
- Is implemented in the user level library, they are not created using the system calls.
- Thread switching does not need to call OS and to cause interrupt to Kernel.
- Examples: Java thread, POSIX threads, etc.
- Advantages of User-level threads
 - The user threads can be easily implemented than the kernel thread.
 - User-level threads can be applied to such types of operating systems that do not support threads at the kernel-level.
 - It is faster and efficient.
 - Context switch time is shorter than the kernel-level threads.
 - It does not require modifications of the operating system.
 - User-level threads representation is very simple. The register, PC, stack, and mini thread control blocks are stored in the address space of the user-level process.
 - It is simple to create, switch, and synchronize threads without the intervention of the process.
- Disadvantages of User-level threads
 - User-level threads lack coordination between the thread and the kernel.
 - If a thread causes a page fault, the entire process is blocked.

Kernel Level Thread

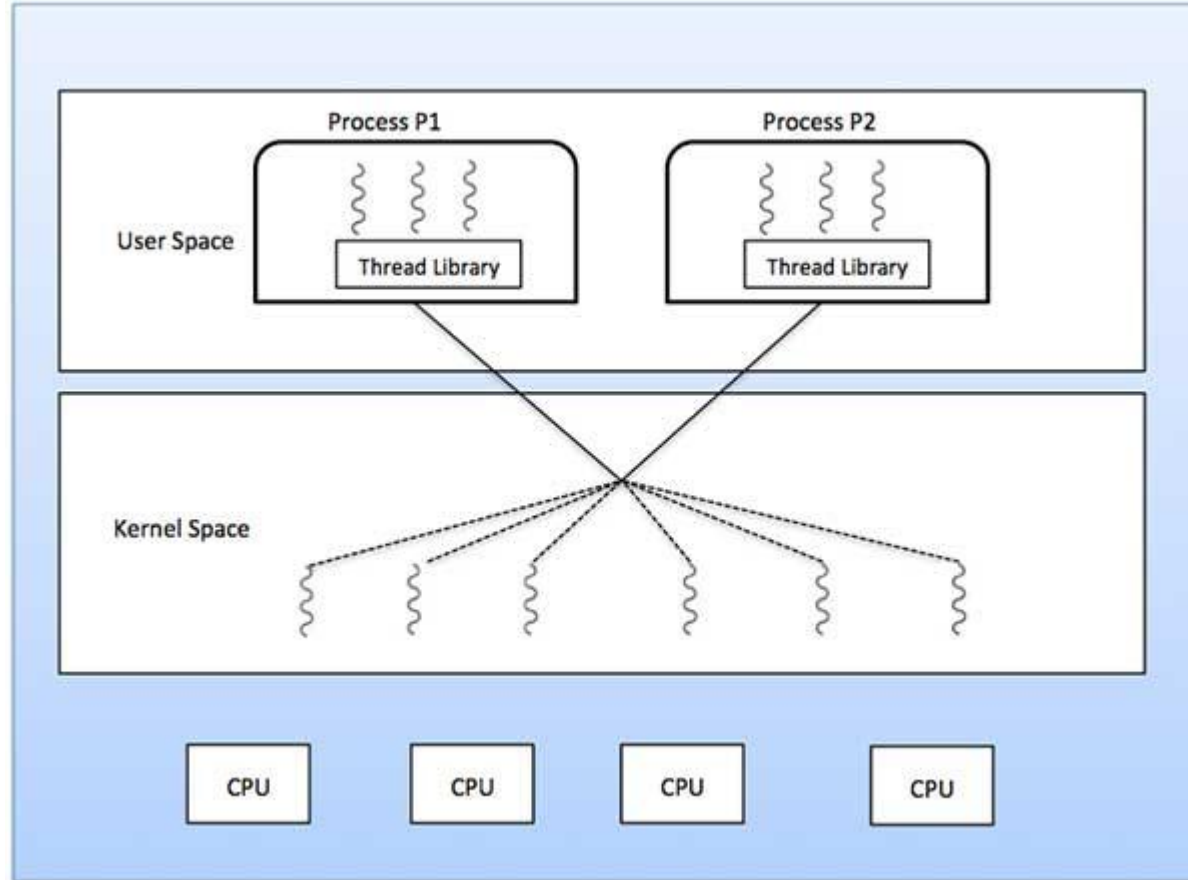
- Kernel knows and manages the threads.
- kernel has thread table (a master one) that keeps track of all the threads in the system.
 - In addition kernel also maintains the traditional process table to keep track of the processes.
- OS kernel provides system call to create and manage threads.
- There is a thread control block and process control block in the system for each thread and process in the kernel-level thread.
- Advantages of Kernel-level threads
 - The kernel-level thread is fully aware of all threads.
 - The scheduler may decide to spend more CPU time in the process of threads being large numerical.
 - The kernel-level thread is good for those applications that block the frequency.
- Disadvantages of Kernel-level threads
 - The kernel thread manages and schedules all threads.
 - The implementation of kernel threads is difficult than the user thread.
 - The kernel-level thread is slower than user-level threads.

Multithreading Models

- Some operating system provide a combined user level thread and Kernel level thread facility.
- In a combined system, multiple threads within the same application can run in parallel on multiple processors
- Three types
 - Many to One
 - One to One
 - Many to Many

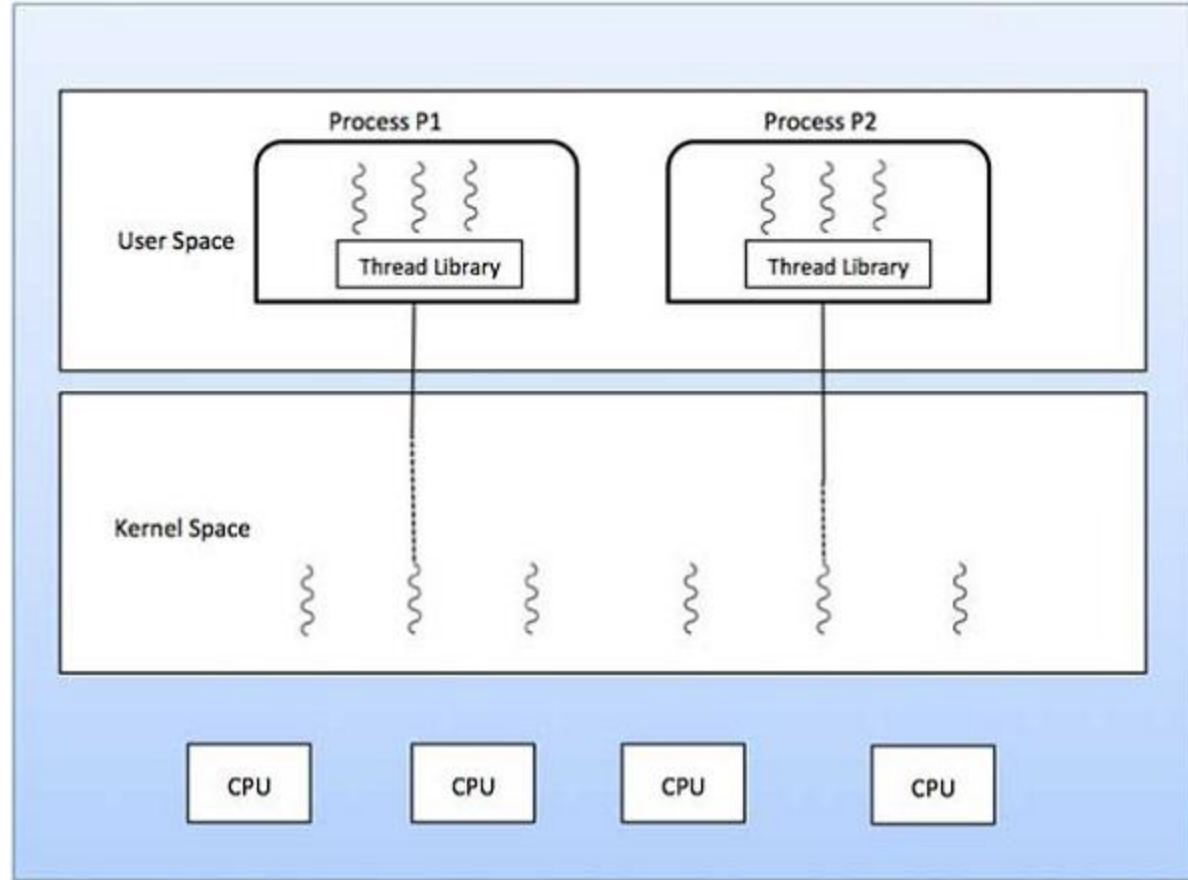
Many to Many Model

- Any number of user threads onto an equal or smaller number of kernel threads.
- Developers can create as many user threads as necessary and the corresponding Kernel threads can run in parallel on a multiprocessor machine.
- Blocking kernel system calls do not block the entire process.
- Processes can be split across multiple processors.
- Individual processes may be allocated variable numbers of kernel threads, depending on the number of CPUs present and other factors.



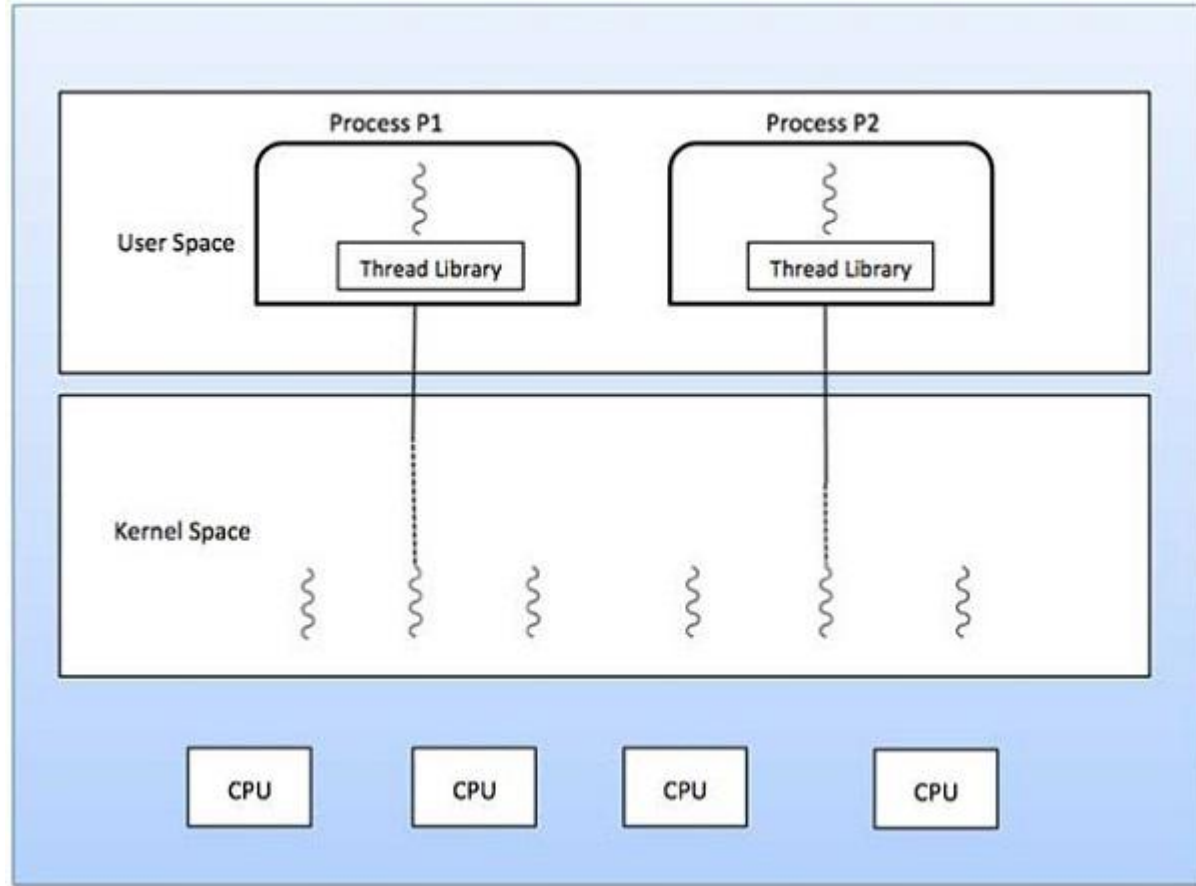
Many to One Model

- Maps many user level threads to one Kernel-level thread.
- Thread management is done in user space by the thread library.
- When thread makes a blocking system call, the entire process will be blocked.
- If the user-level thread libraries are implemented in the operating system in such a way that the system does not support them, then the Kernel threads use the many-to-one relationship modes.
- Green threads for Solaris and GNU Portable Threads implement the many-to-one model in the past, but few systems continue to do so today.



One to One Model

- one-to-one relationship of user-level thread to the kernel-level thread.
- This model provides more concurrency than the many-to-one model.
- It also allows another thread to run when a thread makes a blocking system call.
- It supports multiple threads to execute in parallel on microprocessors.
- Disadvantage of this model is that creating user thread requires the corresponding Kernel thread, involving more overhead and slowing down the system.
- OS/2, windows NT and windows 2000 use one to one relationship model.



Benefits of Thread

- **Enhanced throughput of the system:** When the process is split into many threads, and each thread is treated as a job, the number of jobs done in the unit time increases. That is why the throughput of the system also increases.
- **Effective Utilization of Multiprocessor system:** When you have more than one thread in one process, you can schedule more than one thread in more than one processor.
- **Faster context switch:** The context switching period between threads is less than the process context switching. The process context switch means more overhead for the CPU.
- **Responsiveness:** When the process is split into several threads, and when a thread completes its execution, that process can be responded to as soon as possible.
- **Communication:** Multiple-thread communication is simple because the threads share the same address space, while in process, we adopt just a few exclusive communication strategies for communication between two processes.
- **Resource sharing:** Resources can be shared between all threads within a process, such as code, data, and files. Note: The stack and register cannot be shared between threads. There is a stack and register for each thread.

Scheduler Activation

- Thread Specific Data - Threads belongs to a programme share the data.
 - In some cases, threads need it's own copy of certain.
- Lightweight Process (LWP) - intermediate data structure for communication between user and kernel level thread.
- A virtual processor on which application can schedule an user thread.
- LWP depends on applications
 - CPU bound process need atleast one LWP
 - I/O intensive - multiple LWP
 - Ex - five concurrent request for reading a file - five LWP. If four, one must wait
- Scheduler Activation - Scheme for communication between user and kernel threads
 - Kernel provides an applications with a set of virtual processors (LWP)
 - Application schedule thread user threads there - Upcall
 - Upcall Handler - schedule and respond if blocks